

JacVerify

基于 Jac 与开源 EDA 生态的 图原生智能芯片验证系统

黑客松项目提案与完整 FSM 设计报告

定位: Open-source, graph-native, tool-feedback-driven verification orchestrator

版本 1.0 | 2026-07-25

1. 执行摘要

JacVerify 将芯片验证从一组松散脚本升级为可观测、可恢复、可审计的状态图。系统使用 Jac 表达验证知识图谱和 agent walker，以 Verilator、cocotb、Yosys、SymbiYosys、GTKWave/Surfer 等开源工具提供确定性反馈；LLM 仅负责错误分类、假设生成、测试/断言草拟和下一步策略，不负责宣称设计正确。

项目要素	定义
核心问题	规格、RTL、测试、日志、波形、coverage 与 formal 结果割裂；人工决定下一步，闭环慢且难追踪。
核心方案	把验证生命周期建模为分层 FSM，把验证实体建模为知识图谱，由 Jac walker 根据真实工具反馈驱动状态迁移。
黑客松 MVP	Buggy FIFO + Markdown 规格；自动 lint、compile、cocotb 测试、失败诊断、定向测试生成、重验；formal 为加分项。
关键创新	Graph-native memory、显式 FSM、统一 ToolResult、requirement-to-evidence traceability、受约束的 agent 行为。
最终演示	系统发现 FIFO wraparound 或 simultaneous read/write bug，展示证据、假设、动作、重验结果及需求追踪图。

2. 问题定义与设计原则

- 验证不是一次性代码生成，而是“规格 -> 检查 -> 执行 -> 反馈 -> 诊断 -> 修复 -> 重验”的长循环。
- 工具执行必须确定性、可重放：固定版本、命令、seed、timeout 和 artifact 路径。
- LLM 输出必须结构化并受白名单约束；任何“通过”结论都必须来自工具结果或签核规则。
- 每次状态迁移都记录触发事件、证据、动作、预算消耗和产物，支持恢复与审计。
- 优先做窄而完整的闭环，而不是同时接入很多不能联动的工具。

3. 总体架构

系统由控制平面、知识平面、执行平面和证据平面组成。Jac 位于控制与知识平面，Python adapter 负责与 EDA 工具交互。

层	职责	主要组件
交互层	上传 RTL/规格、启动任务、查看 FSM/图谱/证据	Jac Web/API、简单前端或 CLI
控制平面	规划、状态迁移、预算、重试、人工门禁	Jac walkers、FSM engine、policy guard
知识平面	持久化实体关系与历史验证记忆	Jac nodes/edges、Verification Graph
执行平面	沙箱运行确定性工具并标准化结果	Tool adapters、Docker/OSS CAD Suite
证据平面	保存日志、VCD/FST、coverage、counterexample、patch	Artifact store + metadata index

```
User / API
-> Jac Workflow Controller
  -> Verification Knowledge Graph
  -> State Machine + Policy / Budget
  -> Tool Adapter Layer
    -> Verilator / Icarus / cocotb
    -> Yosys / SymbiYosys / SMT solver
    -> GTKWave / Surfer / coverage parser
  <- ToolResult + artifacts
  <- status, evidence, traceability
```

4. 顶层验证 FSM

顶层 FSM 负责从规格进入签核。任何阶段失败均进入统一诊断闭环；coverage gap 不一定是“失败”，而是触发生成定向测试或断言的规划事件。

状态组	包含状态	迁移语义
主流程	S0-S9	正常路径严格向前；只有设计、TB 或证据变化时才回退到最早受影响阶段。
诊断闭环	D0-D5	失败先归一化，再形成可验证假设；禁止在没有证据的情况下直接修改 RTL。
终止状态	DONE / BLOCKED	DONE 表示所有签核 guard 满足；BLOCKED 表示需要人工信息、审批或预算已经耗尽。
非失败事件	coverage_hole / missing_evidence	触发补测试、补断言或补 formal evidence，不等价于设计 bug。

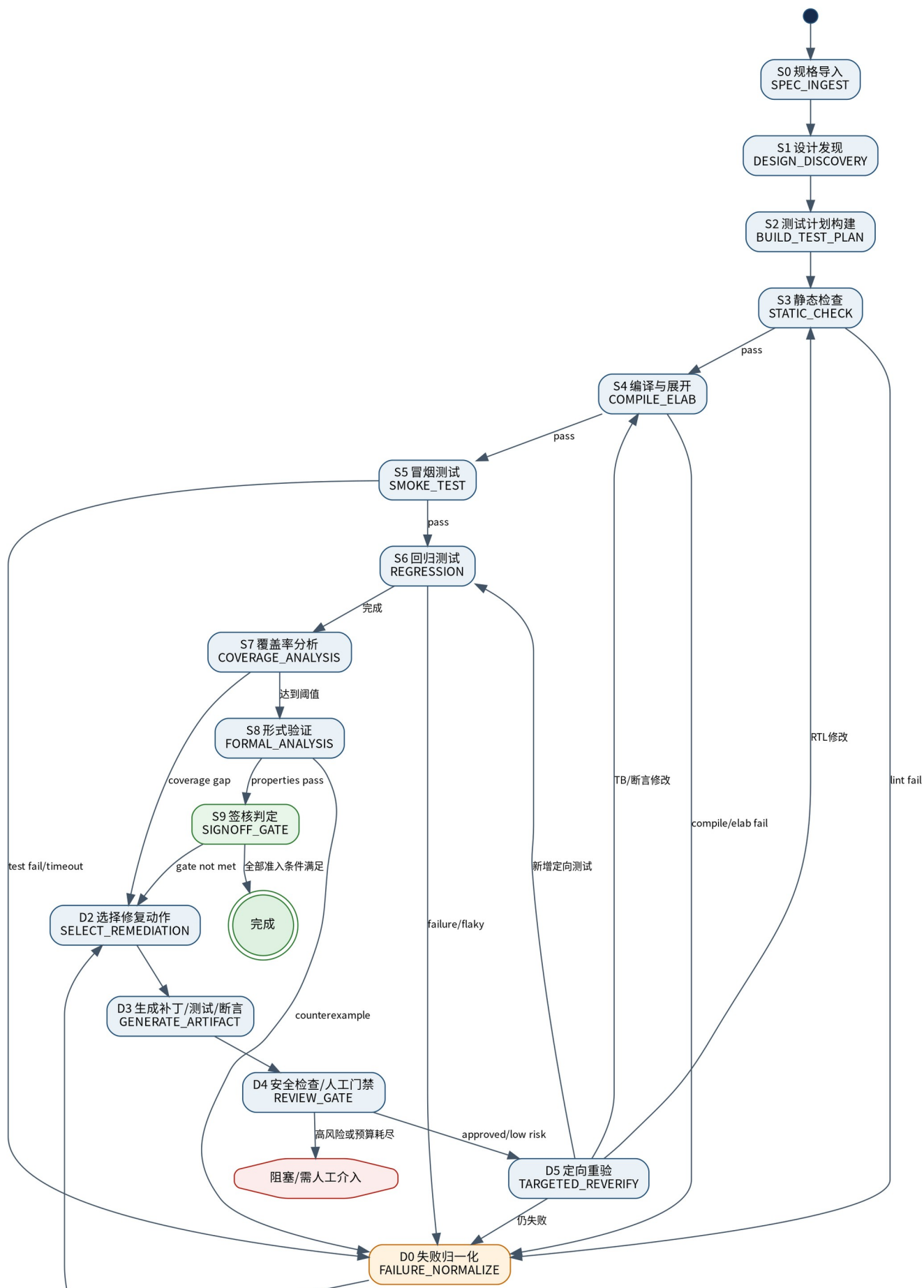


图 1. JacVerify 顶层验证生命周期与调试闭环

ID	状态	职责	成功条件	异常/出口
S0	SPEC_INGEST	读取 Markdown/YAML 规格，拆分 requirement，建立验收条件	规格可解析且至少一个 requirement	规格歧义/缺失 -> BLOCKED
S1	DESIGN_DISCOVERY	扫描 RTL、模块层次、端口、时钟复位、依赖	top/module graph 完整	解析失败 -> D0
S2	BUILD_TEST_PLAN	将 requirement 映射到 test/assertion/coverage intent	每个 requirement 有验证策略	无法验证项 -> 人工门禁
S3	STATIC_CHECK	lint、语法、宽度、未驱动、组合环等	无阻塞级错误	lint fail -> D0
S4	COMPILE_ELAB	编译、展开、构建仿真对象	binary/build artifact 生成	compile/elab fail -> D0
S5	SMOKE_TEST	reset、基本读写、最短路径测试	所有 smoke 通过	失败/超时 -> D0
S6	REGRESSION	多 seed、多场景执行和失败聚类	预算内完成且无未豁免失败	fail/flaky -> D0
S7	COVERAGE_ANALYSIS	计算代码/功能/需求覆盖，识别 hole	达到阈值或 gap 有处置计划	gap -> D2
S8	FORMAL_ANALYSIS	运行 safety/liveness/bounded properties	所有必需 property pass	counterexample -> D0
S9	SIGNOFF_GATE	聚合需求、测试、coverage、formal、豁免	所有 gate 满足	未满足 -> D2
D0	FAILURE_NORMALIZE	将工具输出归类为结构化 Failure	得到 category、location、evidence	证据不足 -> D1 低置信
D1	HYPOTHESIS_RANK	生成并按证据/历史/可验证性排序假设	至少一个可执行 hypothesis	无可行假设 -> BLOCKED
D2	SELECT_REMEDIATION	在 test/assertion/RTL/TB/config 间选动作	动作满足 policy 与预算	高风险 -> D4
D3	GENERATE_ARTIFACT	生成定向测试、断言、补丁或配置变更	产物通过静态安全检查	生成失败 -> D1
D4	REVIEW_GATE	检查修改范围、风险、权限和人工审批	批准或低风险自动通过	拒绝/预算耗尽 -> BLOCKED
D5	TARGETED_REVERIFY	先运行最小复现，再扩展到相关回归	失败消失且无新回归	仍失败 -> D0

5. 完整状态迁移规则

From	To	触发条件	迁移动作
START	S0	job_created	创建 Job/Graph 根节点
S0	S1	spec_valid	保存 Requirement nodes
S0	BLOCKED	spec_ambiguous	输出待澄清项
S1	S2	design_graph_ready	保存模块/端口/依赖
S1	D0	parse_failed	保存 parser log
S2	S3	testplan_ready	创建 TestPlanItem
S2	D4	unverifiable_requirement	人工确认/豁免
S3	S4	lint_pass	记录 lint evidence
S3	D0	lint_fail	Failure(category=lint)
S4	S5	compile_pass	保存 binary/build manifest
S4	D0	compile_fail	Failure(category=compile)
S5	S6	smoke_pass	提升到 regression
S5	D0	smoke_fail timeout	保存 seed/VCD/log
S6	S7	regression_complete	聚合 run 结果
S6	D0	deterministic_fail	失败聚类
S6	D0	flaky_detected	触发重复运行和竞态分析
S7	S8	coverage_gate_met	冻结 coverage snapshot
S7	D2	coverage_hole	生成定向 stimulus/assertion
S8	S9	formal_pass	保存 proof evidence
S8	D0	counterexample	保存 trace/counterexample

From	To	触发条件	迁移动作
S9	DONE	all_signoff_gates_met	生成最终报告
S9	D2	missing_evidence	选择补证据动作
D0	D1	failure_normalized	创建 Failure node
D1	D2	hypothesis_available	创建 ranked Hypothesis
D1	BLOCKED	no_actionable_hypothesis	输出人工 debug package
D2	D3	action_allowed	分配 action budget
D2	D4	high_risk_action	请求审批
D3	D4	artifact_generated	运行 diff/policy checks
D4	D5	approved auto_approved	创建 TargetedRun
D4	BLOCKED	rejected budget_exhausted	保留上下文
D5	S3	rtl_changed	从 lint 重启
D5	S4	tb_or_assertion_changed	从 compile 重启
D5	S6	test_only_changed	执行定向测试后相关回归
D5	D0	failure_persists	更新证据与假设置信度

6. FSM 数据模型、Guard 与预算

```
VerificationContext {
  job_id, current_state, design_revision, toolchain_digest,
  requirements[], testplan[], active_failure?, hypotheses[],
  run_budget, retry_budget, token_budget, wallclock_deadline,
  coverage_thresholds, required_properties[], waivers[], artifacts[]
}

TransitionRecord {
  from_state, to_state, event, guard_result,
  tool_result_id, evidence_ids[], action_id,
  timestamp, attempt, budget_delta
}
```

Guard	示例规则
lint_pass	exit_code == 0 AND blocking_diagnostics == 0
smoke_pass	all(required_smoke_tests.status == PASSED)
coverage_gate_met	line >= 90% AND branch >= 85% AND all requirement intents covered; MVP 可降低阈值
formal_pass	all(required_properties.status == PASSED) AND no UNKNOWN
action_allowed	action in whitelist AND touched_files within scope AND estimated_cost <= remaining_budget
signoff	no unresolved severity>=high failure; requirements 均有 evidence 或显式 waiver

推荐的预算策略：单工具最多重试 2 次；同一失败假设最多 3 个；自动 RTL patch 只允许修改指定模块且必须经过人工门禁；总运行次数和 wall-clock deadline 写入 context。

7. 单次工具执行子 FSM

每一次 lint、compile、simulation 或 formal run 都使用统一子状态机，避免 agent 直接执行不受控 shell。

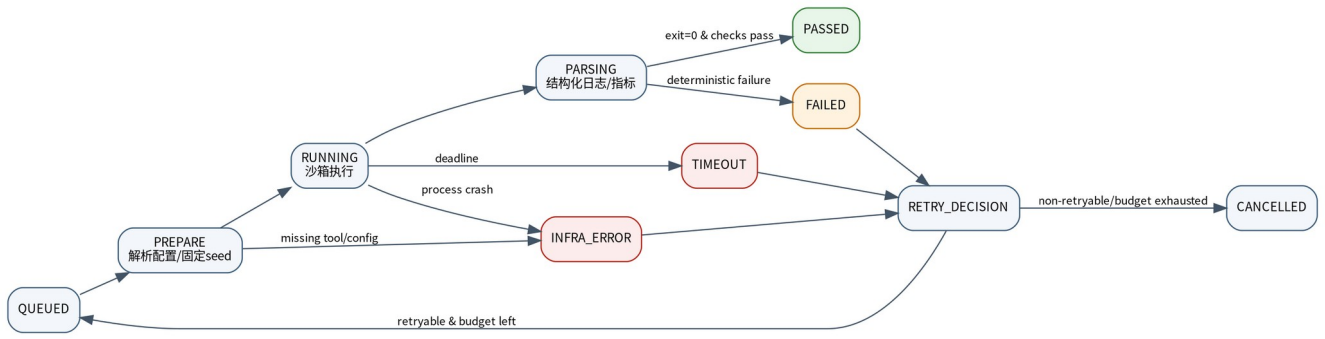


图2. Tool Execution FSM

```

ToolResult {
  tool, version, stage, status, exit_code,
  command[], cwd, seed, start_time, duration_ms,
  stdout_path, stderr_path, artifacts[],
  diagnostics[], metrics{}, retryable
}
  
```

8. 失败诊断与修复子 FSM

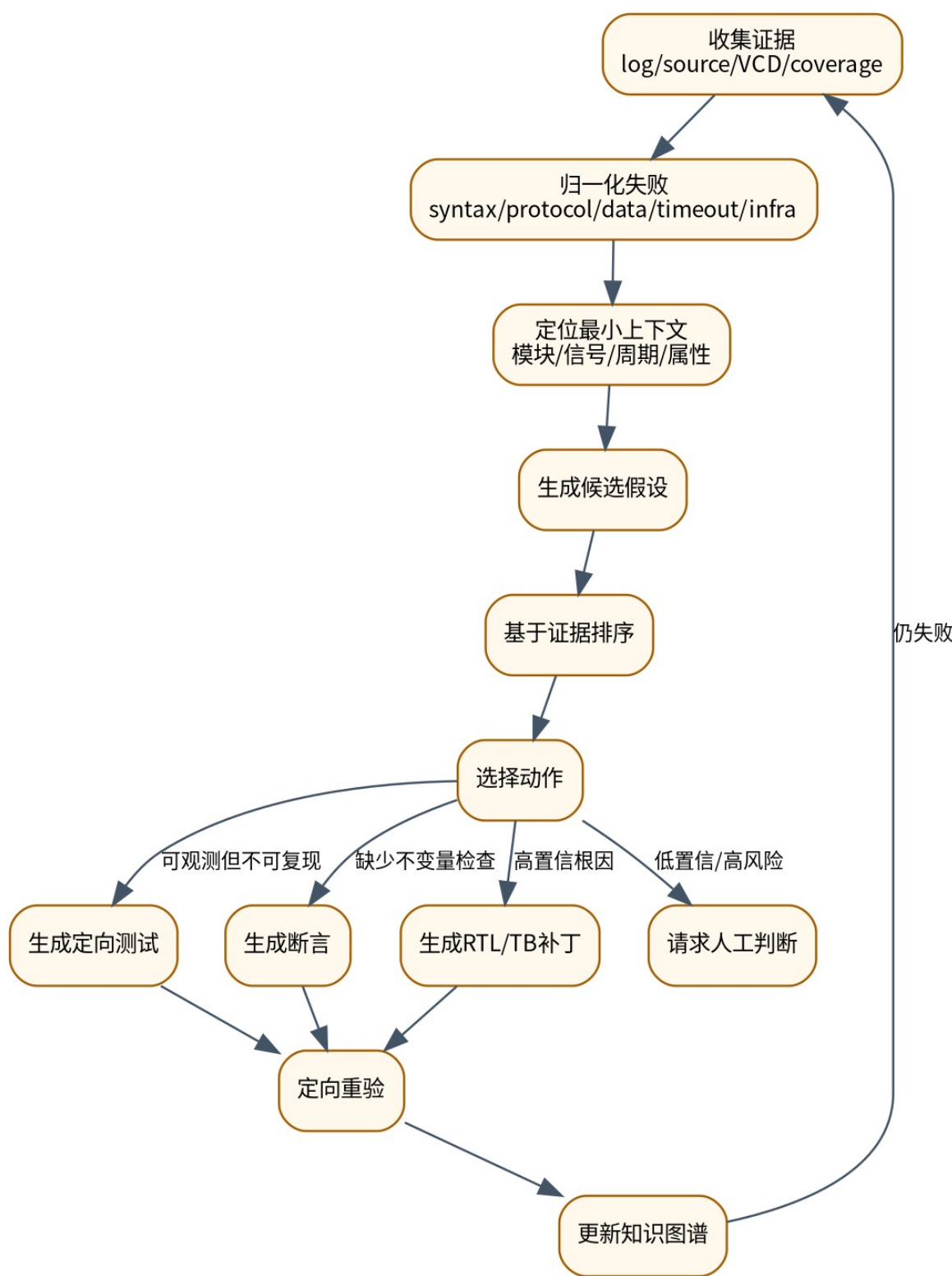


图 3. Evidence-driven Debug FSM

失败类别	典型证据	优先动作
syntax/compile	文件、行号、parser diagnostic	局部修复或回退生成产物

失败类别	典型证据	优先动作
protocol violation	assertion、握手时序、transaction trace	生成定向序列 + protocol assertion
data mismatch	scoreboard expected/actual、首次分歧周期	缩小输入、定位 producer/consumer
timeout/deadlock	无进展窗口、等待关系、最后活跃信号	生成 liveness assertion，检查 ready/valid/clock/reset
formal counterexample	property、最短 trace、初始约束	重放 counterexample，修正设计或约束
coverage hole	未命中分支/交叉/需求	生成约束随机或 directed test
infrastructure	工具缺失、容器崩溃、权限/磁盘	不修改 RTL；修复环境后重试

9. Jac 建模与 Agent 设计

Jac 的 node/edge 用于持久化验证知识；walker 对应状态处理器。FSM 引擎只接受结构化事件，walker 不能任意跳转。

```
node Requirement { has id: str; has text: str; has status: str = "unverified"; }
node Module      { has name: str; has path: str; }
node Test        { has name: str; has kind: str; has status: str = "new"; }
node Run         { has stage: str; has status: str; has seed: int; }
node Failure     { has category: str; has evidence: list; has severity: str; }
node Hypothesis  { has claim: str; has confidence: float; has next_action: str; }
node Artifact    { has uri: str; has digest: str; has kind: str; }
```

```
edge verifies; edge targets; edge produced; edge failed_with;
edge supports; edge suggests; edge modifies; edge derived_from;
```

```
walker state_executor {
  can on_state with VerificationJob entry {
    # 1. 读取 current_state 和 context
    # 2. 调用对应 deterministic adapter / constrained LLM function
    # 3. 生成 Event + Evidence
    # 4. 由 transition table + guard 计算 next_state
    # 5. 持久化 TransitionRecord
  }
}
```

10. 各阶段可接入的开源工具链

阶段	工具	接入产物
规格/需求	Markdown/YAML、JSON Schema	结构化 requirement 与验收条件
SV 解析/层次	Surelog/UHDM、Slang、PyVerilog	AST、module/interface/signal graph
格式与 lint	Verible、Verilator --lint-only	语法、风格、宽度、未使用/未驱动
仿真	Verilator、Icarus Verilog、GHDL	执行 RTL 与生成 VCD/FST
测试平台	cocotb、pytest	Python coroutine driver/monitor/scoreboard
随机/属性测试	Hypothesis、cocotb-coverage	输入生成与功能覆盖
综合 sanity	Yosys	可综合性、netlist、统计
形式验证	SymbiYosys + Yosys	BMC、prove、cover
SMT/SAT	Boolector、Z3、Yices	formal backend
Coverage	Verilator coverage、cocotb-coverage	line/toggle/functional metrics
波形	GTKWave、Surfer	VCD/FST 查看；agent 读取结构化 trace
时序	OpenSTA	静态时序检查，可作为后续扩展
RTL-to-GDS	OpenROAD Flow Scripts	长期扩展到物理实现反馈

阶段	工具	接入产物
环境/CI	OSS CAD Suite、Docker、GitHub Actions	可重放工具链和自动 regression

11. Tool Adapter 接口

```

class ToolAdapter(Protocol):
    def prepare(self, ctx: RunContext) -> PreparedRun: ...
    def execute(self, run: PreparedRun) -> RawExecution: ...
    def parse(self, raw: RawExecution) -> ToolResult: ...

class VerilatorAdapter:
    def lint(self, rtl_files: list[str]) -> ToolResult: ...
    def compile(self, top: str, rtl_files: list[str]) -> ToolResult: ...

class CocotbAdapter:
    def run(self, test: str, seed: int, timeout_s: int) -> ToolResult: ...

class SymbiYosysAdapter:
    def prove(self, sby_file: str, timeout_s: int) -> ToolResult: ...

```

安全规则：命令由 adapter 模板生成；参数经过白名单和路径校验；禁止 LLM 直接拼接 shell；容器只挂载工作区；所有输出按 job/run 分目录保存。

12. 黑客松 MVP 与演示脚本

输入	内容
RTL	带有 pointer width 或 simultaneous read/write bug 的同步 FIFO
规格	5 条自然语言需求：reset、FIFO 顺序、full 禁写、empty 禁读、并发读写 occupancy
测试	cocotb smoke + 自动生成 directed test
工具	Jac、Python、Verilator、cocotb、VCD；SymbiYosys 作为 bonus

1. 上传 fifo.sv 与 fifo_spec.md，系统生成 Requirement 和 Module nodes。
2. 进入 S3，Verilator lint；通过后进入 S4/S5。
3. 基础 smoke test 通过，regression 在 wraparound 场景失败。
4. D0 从 scoreboard mismatch、seed、cycle 和 VCD 建立 Failure。
5. D1 给出假设：“write pointer 额外 wrap bit 缺失”或“并发读写 occupancy 更新优先级错误”。
6. D2/D3 选择生成 DEPTH+2 的定向测试或 assertion，并创建产物。
7. D5 运行最小复现；若测试稳定复现，展示 patch suggestion；若已授权可应用补丁，再从 S3 重验。
8. Coverage 与 requirement traceability 页面显示：每条需求对应 test/assertion/run/evidence。

13. 签核与演示成功标准

类别	MVP Gate
可执行性	一条命令启动；工具版本和 seed 可见；失败可重放
FSM	UI/CLI 显示当前状态、上次事件、下一步和迁移原因
闭环	至少完成一次“发现失败 -> 诊断 -> 生成定向测试/修复 -> 重验”
证据	保存日志、VCD、测试结果和结构化 ToolResult
追踪性	至少 5 个 requirements 都能连接到 test/assertion/run
安全性	LLM 不直接运行任意 shell；RTL 修改需要门禁
演示稳定性	预置固定 bug、固定 seed 和 fallback 录制 artifact

14. 风险与取舍

风险	处理
黑客松时间不足	只接 Verilator+cocotb；formal、UI、自动 patch 依次作为 bonus。
LLM 输出不稳定	使用 JSON schema、动作白名单、固定 demo case，并缓存一次成功结果。
波形太大难分析	MVP 只提取失败前后有限周期和关键信号，不把完整 VCD 放进 prompt。
“全自动修 RTL” 风险高	主 demo 生成 test/assertion 和 patch suggestion；应用 RTL patch 设置人工门禁。
开源 SystemVerilog 支持差异	MVP 限制为可综合 SV 子集，并准备 Icarus/Verilator fallback。
创新点被误解为脚本包装	重点展示 graph memory、显式迁移记录、跨工具 evidence 和 requirement traceability。

15. 实施计划

阶段	交付物
0-2 小时	FIFO 设计/规格、Docker 或本机工具链、手工 cocotb baseline
2-5 小时	ToolResult、Verilator/cocotb adapters、artifact 目录规范
5-8 小时	Jac nodes/edges、顶层 FSM 和 transition logger
8-12 小时	失败 parser、LLM 结构化假设、定向测试生成
12-16 小时	状态/证据/图谱展示与完整 demo rehearsal
Bonus	SymbiYosys property、自动 patch 门禁、coverage hole planner

16. 90 秒 Pitch

芯片验证不是一次代码生成，而是规格、测试、仿真、覆盖率、形式验证、失败诊断和重验组成的持续反馈循环。今天这些步骤通常由脚本和人工判断连接，AI coding agent 也缺少对“验证了什么、还缺什么、上次为什么失败”的持久化理解。

JacVerify 是一个基于 Jac 的图原生智能验证编排器。我们把 requirement、module、test、run、coverage hole、failure、hypothesis 和 patch 建模为知识图谱，并把整个验证生命周期实现为显式 FSM。Jac walker 根据 Verilator、cocotb、Yosys 和 SymbiYosys 的真实反馈决定下一步；LLM 只做受约束的语义任务，绝不自行宣称设计正确。

在演示中，系统读取一个带 bug 的 FIFO 和自然语言规格，自动运行检查与测试，发现 wraparound 失败，关联日志、波形

和需求，生成排序后的根因假设及定向测试，然后重验并展示完整 requirement-to-evidence traceability。JacVerify 将芯片验证从零散脚本变成可观测、可恢复、可审计的 agentic workflow。

附录 A：推荐动作白名单

```
RUN_PARSE, RUN_LINT, RUN_COMPILE, RUN_SMOKE,  
RUN_TEST, RUN_REGRESSION, ANALYZE_COVERAGE,  
RUN_FORMAL, ANALYZE_FAILURE, GENERATE_TEST,  
GENERATE_ASSERTION, SUGGEST_PATCH, APPLY_PATCH_WITH_APPROVAL,  
TARGETED_REVERIFY, REQUEST_HUMAN_REVIEW, COMPLETE
```

附录 B：目录结构建议

```
jacverify/  
  app.jac  
  graph/          # nodes, edges, walkers  
  fsm/            # states, transitions, guards  
  adapters/       # verilator.py, cocotb.py, sby.py  
  parsers/        # diagnostics, junit, coverage, VCD slices  
  policies/       # command whitelist, patch scope, budgets  
  demo/fifo/      # rtl, spec, tests, properties  
  runs/<job>/<run>/ # command, logs, artifacts, ToolResult.json  
  ui/             # optional status/graph dashboard
```